

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Lista.h"
4
5  Celula* criarLista() {
6      return NULL;
7  }
8
9  int listaVazia(Celula* sequencia) {
10     return (sequencia == NULL);
11 }
12
13 void inserirInicio(Celula** sequencia, int numero) {
14     Celula* novoNo = (Celula*)malloc(sizeof(Celula));
15     if (novoNo == NULL) return;
16     novoNo->dado = numero;
17     novoNo->proximo = *sequencia;
18     *sequencia = novoNo;
19 }
20
21 void inserirFim(Celula** sequencia, int numero) {
22     Celula* novoNo = (Celula*)malloc(sizeof(Celula));
23     if (novoNo == NULL) return;
24     novoNo->dado = numero;
25     novoNo->proximo = NULL;
26     if (listaVazia(*sequencia)) {
27         *sequencia = novoNo;
28         return;
29     }
30     Celula* auxiliar = *sequencia;
31     while (auxiliar->proximo != NULL) {
32         auxiliar = auxiliar->proximo;
33     }
34     auxiliar->proximo = novoNo;
35 }
36
37 void mostrarLista(Celula* sequencia) {
38     if (listaVazia(sequencia)) {
39         printf("[Lista Vazia]\n");
40         return;
41     }
42     Celula* auxiliar = sequencia;
43     while (auxiliar != NULL) {
44         printf("%d -> ", auxiliar->dado);
45         auxiliar = auxiliar->proximo;
46     }
47     printf("NULL\n");
48 }
49
50 void liberarLista(Celula** sequencia) {
51     Celula* auxiliar = *sequencia;
52     while (auxiliar != NULL) {
53         Celula* proxNo = auxiliar->proximo;
54         free(auxiliar);
55         auxiliar = proxNo;
56     }
57     *sequencia = NULL;
58 }
59
60 int maxLista(Node* lista) {
61     if (listaVazia(lista)) return -1;
62     int maiorValor = lista->dado;
63     Node* auxiliar = lista->proximo;
64     while (auxiliar != NULL) {
65         if (auxiliar->dado > maiorValor) {
66             maiorValor = auxiliar->dado;

```

```

67     }
68     auxiliar = auxiliar->proximo;
69 }
70 return maiorValor;
71 }
72
73 int minLista(Node* lista) {
74     if (listaVazia(lista)) return -1;
75     int menorValor = lista->dado;
76     Node* auxiliar = lista->proximo;
77     while (auxiliar != NULL) {
78         if (auxiliar->dado < menorValor) {
79             menorValor = auxiliar->dado;
80         }
81         auxiliar = auxiliar->proximo;
82     }
83     return menorValor;
84 }
85
86 float mediaLista(Node* lista) {
87     if (listaVazia(lista)) return 0.0;
88     int acumuladorSoma = 0;
89     int contadorItens = 0;
90     Node* auxiliar = lista;
91     while (auxiliar != NULL) {
92         acumuladorSoma += auxiliar->dado;
93         contadorItens++;
94         auxiliar = auxiliar->proximo;
95     }
96     return (float)acumuladorSoma / contadorItens;
97 }
98
99 int lenLista(Node* lista) {
100     int tamanho = 0;
101     Node* auxiliar = lista;
102     while (auxiliar != NULL) {
103         tamanho++;
104         auxiliar = auxiliar->proximo;
105     }
106     return tamanho;
107 }
108
109 Nodo* appendLista(Nodo* L1, Nodo* L2) {
110     if (listaVazia(L1)) return L2;
111     Nodo* auxiliar = L1;
112     while (auxiliar->proximo != NULL) {
113         auxiliar = auxiliar->proximo;
114     }
115     auxiliar->proximo = L2;
116     return L1;
117 }
118
119 Nodo* reverseLista(Nodo* lista) {
120     Nodo* anterior = NULL;
121     Nodo* atual = lista;
122     Nodo* proximoNo = NULL;
123     while (atual != NULL) {
124         proximoNo = atual->proximo;
125         atual->proximo = anterior;
126         anterior = atual;
127         atual = proximoNo;
128     }
129     return anterior;
130 }
131
132 int checarSequencia(Nodo* pPrincipal, Nodo* pSub) {

```

```
133     while (pSub != NULL) {
134         if (pPrincipal == NULL || pPrincipal->dado != pSub->dado) {
135             return 0;
136         }
137         pPrincipal = pPrincipal->proximo;
138         pSub = pSub->proximo;
139     }
140     return 1;
141 }
142
143 int isSubLista(Nodo* L1, Nodo* L2) {
144     if (listaVazia(L2)) return 1;
145     if (listaVazia(L1)) return 0;
146     Nodo* auxiliarP = L1;
147     while (auxiliarP != NULL) {
148         if (auxiliarP->dado == L2->dado) {
149             if (checarSequencia(auxiliarP, L2)) {
150                 return 1;
151             }
152         }
153         auxiliarP = auxiliarP->proximo;
154     }
155     return 0;
156 }
```